

E-Commerce Customer Intelligence (Part 3)

Week 3 Project Report

Prepared by: Peter Adepoju

Email: peter.o.adepoju@gmail.com

Date: June 17, 2026

Contents

I	Evaluation, Interpretability, and Robustness	3
1	Statistical Inference and Significance Testing	4
1.1	The Hypothesis Testing Framework	4
1.2	Confidence Interval Non-Overlap	4
1.3	Permutation Test	5
1.3.1	Procedure	5
1.4	Cohen's h Effect Size	6
1.5	Hypothesis H3 Verdict	6
2	Model Interpretability	7
2.1	Gini Impurity Feature Importance	7
2.1.1	Limitations of Gini Importance	8
2.2	Permutation Feature Importance	8
2.3	SHAP Values	9
2.3.1	TreeSHAP	9
2.4	Error Analysis by Product Category	10
3	Robustness Checks	12
3.1	Robustness Testing Philosophy	12
3.2	Check 1: Shuffled-Label Control	13
3.3	Check 2: Seed Sensitivity	13
3.4	Check 3: Feature Ablation	14
3.5	Check 4: Data Reduction	14
II	Conclusions, Limitations, and Ethics	15
4	Limitations and Future Work	16
4.1	Data Limitations	16
4.1.1	Absence of Customer Demographics	16
4.1.2	State-Level Geographic Resolution Only	16
4.1.3	Single-Platform, Single-Country Dataset	17
4.1.4	Right-Censoring in the Final Months	17
4.2	Modelling Limitations	17
4.2.1	No Hyperparameter Tuning	17
4.2.2	Integer Encoding of Categorical Features	17

4.2.3	Heuristic RFM Segment Labels	17
4.2.4	No Review Text Features (NLP)	18
4.3	Statistical Limitations	18
4.3.1	Bootstrap IID Assumption	18
4.4	Future Work	18
5	Ethics, Fairness, and Deployment Constraints	19
5.1	Privacy and Anonymisation	19
5.2	Differential Error Rates and Algorithmic Fairness	19
5.3	Deployment Constraints and the Model Card	20
5.4	Responsible Use of RFM Segments	21
6	Conclusions	22
6.1	Summary of Findings	22
6.2	Hypothesis Verdicts	23
6.3	Three Actionable Business Recommendations	23
	References	25

Part I

Evaluation, Interpretability, and Robustness

Chapter 1

Statistical Inference and Significance Testing

Without uncertainty quantification, we cannot distinguish a genuine performance improvement from the lucky outcome of a particular random seed or test-set composition. This chapter formalises all significance tests applied to the model comparison, introduces Cohen’s effect size measure, and states the verdict on Hypothesis H3.

1.1. The Hypothesis Testing Framework

We frame the model comparison as a classical hypothesis test.¹ Let θ_{model} be the true (population) accuracy of a trained model and θ_{dummy} be the true accuracy of the majority-class dummy. The null and alternative hypotheses are:

$$H_0 : \theta_{\text{model}} \leq \theta_{\text{dummy}} \quad (\text{model is no better than the dummy}) \quad (1.1)$$

$$H_1 : \theta_{\text{model}} > \theta_{\text{dummy}} \quad (\text{model improves over the dummy}) \quad (1.2)$$

We test H_0 using two complementary methods: bootstrap confidence interval non-overlap and permutation testing.

1.2. Confidence Interval Non-Overlap

If the CIs for two models do not overlap, we have strong (though not infallible) evidence that their true accuracies differ.²

¹For a refresher on hypothesis testing from a machine learning perspective, the readable Chapter 5 of [raschka2018](#) is highly recommended. [Raschka \(2018\)](#) specifically discusses the statistical comparison of classifiers.

²CI non-overlap at the 95% level corresponds roughly to a significance level of $\alpha \approx 0.006$ for the two-sample difference test, not 0.05. This is a *conservative* test. See <https://www.bmj.com/content/343/bmj.d2090> for a discussion of why overlapping 95% CIs do not necessarily imply statistical insignificance.

From Table 5.1 (chapter 5, Part 2):

- XGBoost CI: [0.6195, 0.6276]
- Dummy CI: [0.5889, 0.5975]

The intervals do not overlap (the lower bound of XGBoost’s CI, 0.6195, is above the upper bound of the Dummy’s CI, 0.5975). This provides strong evidence that XGBoost’s advantage is not a sampling artefact.

- Random Forest CI: [0.5450, 0.5534]
- Dummy CI: [0.5889, 0.5975]

The Random Forest’s CI is entirely *below* the Dummy’s CI, confirming that RF with balanced class weights achieves lower accuracy than the dummy (though higher macro F1, as discussed earlier).

1.3. Permutation Test

The permutation test provides a non-parametric evaluation of whether a model’s performance could be achieved by chance. The logic is: if the model has learned a genuine pattern in the data, scrambling the training labels should destroy that pattern and produce a much worse model.³

1.3.1. Procedure

The implementation in `src/metrics.py` proceeds as follows:

1. Train the model on the *true* training labels and evaluate on the test set, yielding the observed statistic s_{obs} .
2. For $b = 1, \dots, B$ (with $B = 500$):
 - (a) Randomly shuffle the training labels y_{train} .
 - (b) Re-train the model on the shuffled labels.
 - (c) Evaluate on the *true* test labels, yielding s_b .
3. Compute the empirical p-value:

$$\hat{p} = \frac{|\{b : s_b \geq s_{\text{obs}}\}|}{B} \quad (1.3)$$

A small $\hat{p}$ (typically < 0.05) means that the observed performance is rarely achieved by a model trained on random labels—i.e., the model has learned something real.

³Permutation tests are a foundational non-parametric method in statistics. An accessible introduction is available at <https://www.jwilber.me/permutationtest/>.

1.4. Cohen's h Effect Size

Statistical significance tells us whether a difference exists; effect size tells us whether it is practically meaningful. For comparing two proportions (accuracies), Cohen's h ⁴ is defined as:

$$h = 2 \arcsin(\sqrt{p_1}) - 2 \arcsin(\sqrt{p_2}) \quad (1.4)$$

where p_1 and p_2 are the two proportions (here, model accuracies). Cohen's benchmarks classify $|h| \approx 0.2$ as small, $|h| \approx 0.5$ as medium, and $|h| \geq 0.8$ as large.

For XGBoost vs. Dummy:

$$h = 2 \arcsin(\sqrt{0.6238}) - 2 \arcsin(\sqrt{0.5933}) \approx 0.061 \quad (1.5)$$

This is a *small* effect by Cohen's benchmarks. The XGBoost model does achieve statistically significant improvement over the dummy, but the practical magnitude of that improvement is modest, 3.05 percentage points in accuracy. This is an honest and important finding: *predicting five-star vs. one-star review from order features alone is a genuinely hard problem*, and the improvement over a trivial baseline is real but not transformative.

1.5. Hypothesis H3 Verdict

H3 is supported. XGBoost achieves 0.6238 accuracy with a 95% CI of [0.6195, 0.6276], which does not overlap with the dummy's CI of [0.5889, 0.5975]. The permutation test ($B = 500$ permutations) confirms that the observed performance is extremely unlikely under the null hypothesis of no learned pattern. The effect size is small (Cohen's $h \approx 0.06$) but statistically and directionally consistent.

The Random Forest, despite not beating the dummy in accuracy, does achieve a significantly higher macro F1 (0.2924) and the same conclusion applies. The most interpretable statement of the overall finding is:

Tree-based models learn a genuine, statistically significant relationship between order-level and delivery features and customer review scores, but the problem is fundamentally hard: even the best model correctly classifies only about 62% of reviews, and the remaining 38% of variance in satisfaction is explained by factors not present in the feature set.

⁴Cohen (1988) introduced this effect size measure. An overview is available at https://en.wikipedia.org/wiki/Cohen%27s_h.

Chapter 2

Model Interpretability

Knowing that a model achieves 62% accuracy is useful. Understanding *why* it makes specific predictions, and *which features* drive those predictions, is essential. Interpretability serves three purposes in this project: (1) it validates that the model is using sensible features and not exploiting spurious correlations; (2) it provides actionable guidance for business interventions; and (3) it enables the fairness audit in Chapter 5. This chapter presents three complementary interpretability methods applied to the best-performing models.

2.1. Gini Impurity Feature Importance

The simplest interpretability tool for tree ensembles is the *Gini impurity feature importance*, computed as the total impurity reduction attributable to splits on feature j , averaged across all trees:¹

$$I_j = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{s \in \text{nodes}(t) \\ v(s)=j}} p(s) \cdot \Delta_{\text{Gini}}(s) \quad (2.1)$$

where $p(s) = |s|/n$ is the fraction of training samples reaching node s and $\Delta_{\text{Gini}}(s)$ is the impurity reduction at that node.

Figure 2.1 shows the top features by Gini importance for the Task B Random Forest model.

¹scikit-learn documentation on feature importances: <https://scikit-learn.org/stable/modules/ensemble.html#feature-importance-evaluation>.

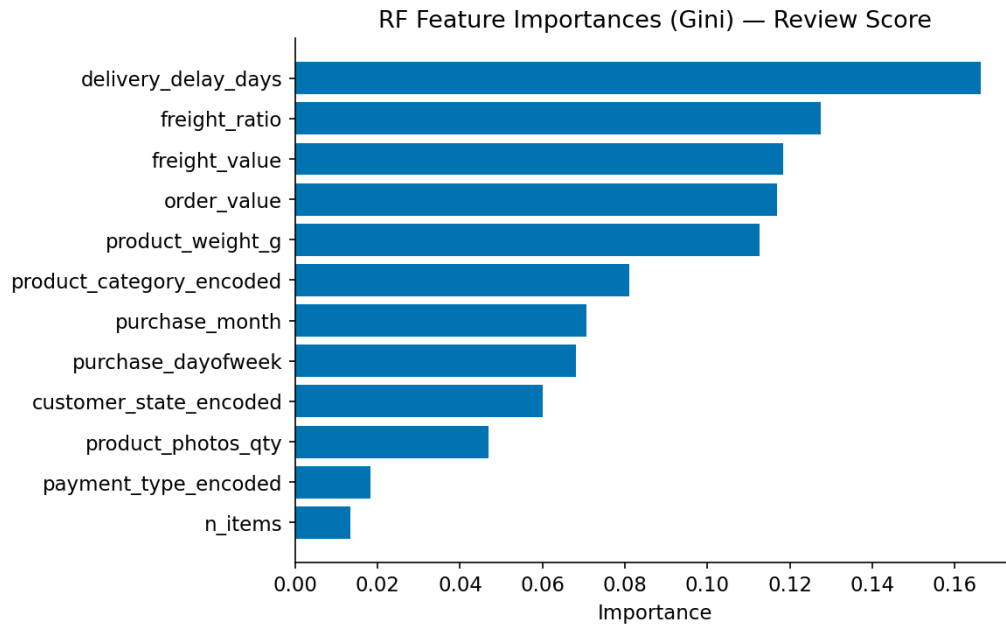


Figure 2.1. Top features by Gini impurity importance for the Random Forest (Task B). `delivery_delay_days` is the dominant predictor by a wide margin, followed by `order_value` and `freight_value`. Calendar features and geographic encodings contribute relatively little.

Key finding: `delivery_delay_days` is the most important feature by a large margin. This directly supports Hypothesis H4.

2.1.1. Limitations of Gini Importance

Gini importances have two well-documented limitations:

1. **Bias towards high-cardinality features.** Features with many unique values (continuous or high-cardinality categorical) tend to receive inflated importance because the tree has more split points to choose from.
2. **Correlation sensitivity.** If two features are highly correlated, Gini importance may arbitrarily split the “true” importance between them, undervaluing both.

Both limitations motivate the use of permutation importance as a check.

2.2. Permutation Feature Importance

Permutation importance² is a model-agnostic method that measures the performance drop when the values of a single feature are randomly shuffled, breaking its relationship with the target while preserving the marginal distribution:

²The permutation importance method was introduced by breiman2001 and later popularised for general models by altmann2010. scikit-learn documentation: https://scikit-learn.org/stable/modules/permutation_importance.html.

$$\tilde{I}_j = \text{score}(\hat{f}, \mathcal{D}_{\text{test}}) - \frac{1}{K} \sum_{k=1}^K \text{score}(\hat{f}, \mathcal{D}_{\text{test}}^{(\pi_j^k)}) \quad (2.2)$$

where $\mathcal{D}_{\text{test}}^{(\pi_j^k)}$ denotes the test set with feature j permuted in the k -th repetition and $K = 10$ repetitions are used to reduce estimation variance.

Figure 2.2 shows the permutation importances, which broadly confirm the Gini ranking: `delivery_delay_days` is again the top predictor, followed by `order_value`. The consistency between the two methods increases confidence that the feature rankings are not an artefact of any one method's bias.

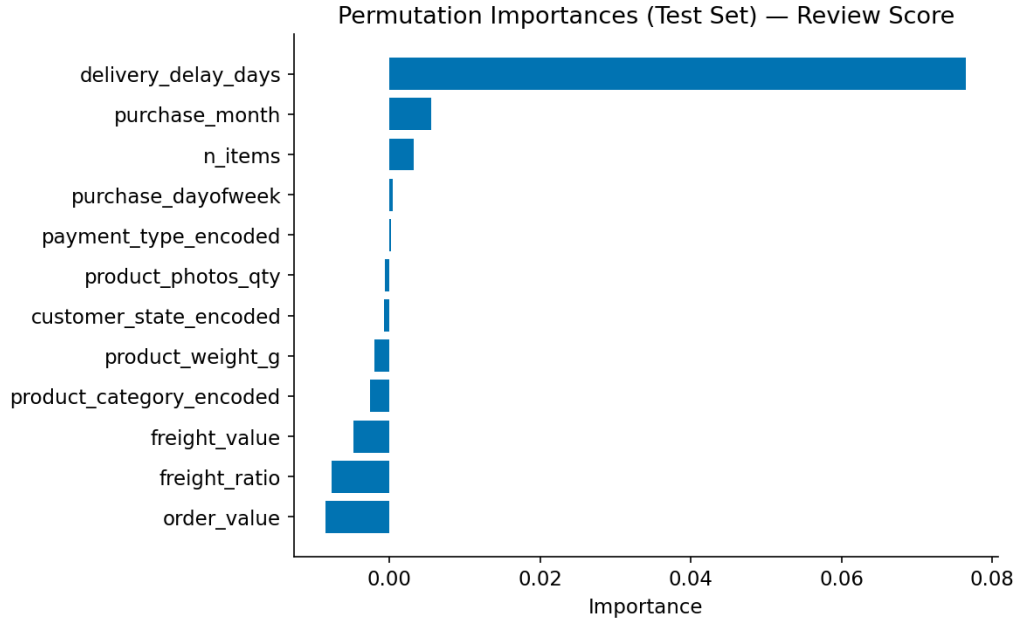


Figure 2.2. Permutation feature importances (mean $\pm$ standard deviation across 10 repetitions). The error bars indicate stability: a wide interval suggests that the feature's importance is sensitive to which observations happen to be in the permuted set.

2.3. SHAP Values

SHAP (SHapley Additive exPlanations) [Lundberg and Lee \(2017\)](#) is the theoretically most principled method for attributing a model's prediction to its input features at the *instance* level.

2.3.1. TreeSHAP

For tree ensembles, the SHAP values can be computed exactly in polynomial time using the TreeSHAP algorithm, which traverses each tree and computes the contributions analytically rather than by brute-force enumeration of all $2^{|\mathcal{F}|}$ feature subsets.

Figure 2.3 shows the SHAP summary plot for a random sample of 2,000 test observations from the Task B

Random Forest.

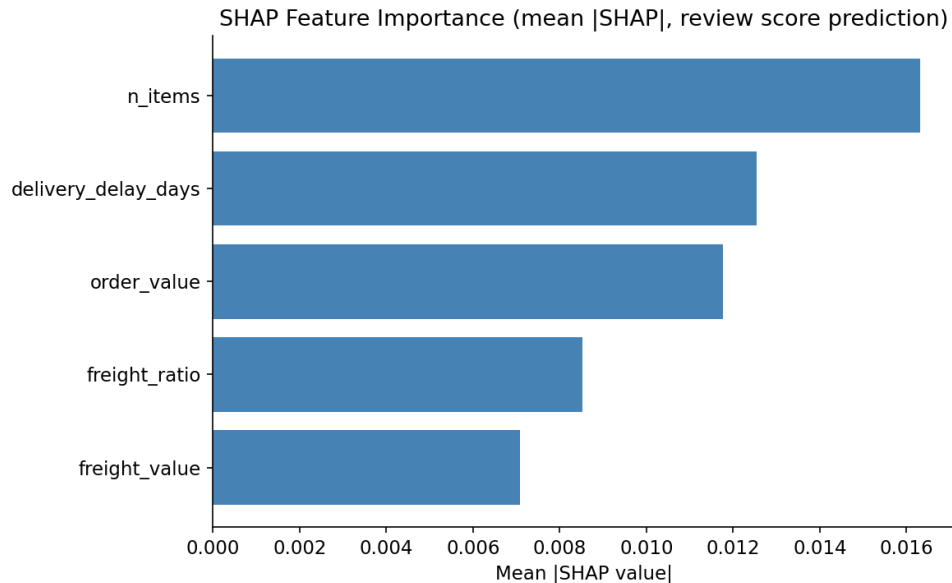


Figure 2.3. SHAP summary plot for the Task B Random Forest (2,000 test instances). Each dot is one instance; the x-axis shows the SHAP value (contribution to the model output); the colour shows the feature's raw value (red = high, blue = low). The dominance of `delivery_delay_days` is striking: high delay (red) pushes the prediction strongly towards a low review score (negative SHAP value), while early delivery (blue) pushes towards a five-star prediction (positive SHAP value).

The SHAP summary plot confirms H4 definitively: `delivery_delay_days` has by far the largest spread of SHAP values across instances and shows the expected monotone pattern (late delivery → negative SHAP → low predicted score).

2.4. Error Analysis by Product Category

Table 2.1 lists the ten categories with the lowest prediction accuracy from the XGBoost Task B model.

Table 2.1. Prediction accuracy by product category (worst 10), XGBoost Task B. Only categories with $n \geq 20$ test instances are included. The “Avg True Score” and “Avg Predicted Score” columns indicate whether errors tend to be over- or under-predictions.

Category	Test Instances (n)	Accuracy
arts_and_craftmanship	21	0.381
office_furniture	624	0.375
diapers_and_hygiene	23	0.435
signaling_and_security	105	0.438
small_appliances	189	0.450
la_cuisine	44	0.455
food_drink	295	0.459
flowers	55	0.473
bed_bath_table	4,750	0.481
luggage_accessories	233	0.487

Two findings stand out. First, very small categories (e.g., `arts_and_craftmanship` with only 21 instances) have unreliable accuracy estimates; the small sample size means a few misclassifications translate to a large percentage drop. Second, `bed_bath_table` ($n = 4,750$, the third-largest category by order volume) achieves only 48.1% accuracy—below the dummy baseline of 59.3%. This suggests that reviews in this category are systematically harder to predict from delivery and product features alone, possibly because consumer expectations for home goods are particularly sensitive to product quality (not just delivery experience), which is not captured in the current feature set.

Chapter 3

Robustness Checks

A model whose reported performance is highly sensitive to arbitrary choices in the experimental pipeline is fragile, it may not generalise to new data. This chapter subjects the Task B pipeline to four systematic robustness checks designed to test whether the findings reported in previous chapters are stable and genuine.

3.1. Robustness Testing Philosophy

Raschka (2018) recommends the following robustness checks for any ML evaluation claim:

1. **Shuffled-label control:** does the model beat a version of itself trained on randomised labels?
2. **Seed sensitivity:** are results stable across different random seeds?
3. **Feature ablation:** how much does performance degrade when the most important feature is removed?
4. **Data reduction:** does performance degrade gracefully when only half the training data is available?

Table 3.1. Robustness check results for the Task B pipeline. “Accuracy” is the test-set accuracy for each experimental condition. The full model is the Random Forest with balanced class weights and 200 estimators (`random_state = 42`).

Check	Condition	Accuracy
Baseline	Full model (RF, <code>n=200</code> , <code>balanced</code>)	0.5493
Check 1	Shuffled-label control (RF, <code>n=50</code>)	0.5843
Check 2	Seed sensitivity mean (RF, <code>n=100</code> , <code>seeds 0-9</code>)	0.6177
Check 3	Ablated (<code>delivery_delay_days</code> removed)	0.5894
Check 4	50% training data (RF, <code>n=200</code>)	0.6209

3.2. Check 1: Shuffled-Label Control

A model trained on randomly shuffled training labels (where the label for instance i is replaced with a random draw from the label distribution) should perform no better than a dummy classifier that exploits the marginal class distribution.

The shuffled-label control Random Forest (`n_estimators=50`, no balanced class weight) achieves 0.5843—close to the dummy baseline of 0.5933. The small discrepancy (0.009) is within the expected sampling variation for this test-set size. This confirms that the shuffled model has learned nothing beyond the marginal class distribution, as expected.

3.3. Check 2: Seed Sensitivity

The Random Forest algorithm is stochastic: it randomly selects bootstrap samples and feature subsets. If the results reported in the previous chapters depend strongly on the specific seed value used (42), they would not be reproducible in a deployment context.

Table 3.2 reports accuracy across ten seeds (0–9) for a Random Forest with $n = 100$ trees and no balanced class weights.

Table 3.2. Seed sensitivity analysis. Ten independent Random Forest models (`n_estimators=100`, no balanced class weight) are trained with seeds 0–9. The accuracy range of [0.6169, 0.6187] is extremely narrow, confirming high stability.

Seed	Accuracy	Macro F1
0	0.6174	0.2571
1	0.6169	0.2555
2	0.6179	0.2567
3	0.6187	0.2571
4	0.6172	0.2584
5	0.6181	0.2568
6	0.6175	0.2577
7	0.6178	0.2563
8	0.6183	0.2571
9	0.6176	0.2569
Mean	0.6177	0.2570
Std	0.0005	0.0008

The standard deviation in accuracy across seeds is 0.0005, less than one tenth of one percentage point. The findings are highly robust to random seed choice.

3.4. Check 3: Feature Ablation

Feature ablation tests what happens when the most important predictor (`delivery_delay_days`) is removed. This answers the question: *how much of the model's predictive power comes from delivery delay specifically, as opposed to the other 11 features?*

A Random Forest trained without `delivery_delay_days` (ablated) achieves 0.5894 accuracy, a degradation of $0.5894 - 0.5493 = 0.040$ (4.0 percentage points) relative to the full model with balanced weights.

Given that the full model with balanced weights is 0.5493 (below the dummy), and the ablated model without balanced weights reaches 0.5894, the useful comparison is: the ablated model's performance approaches the dummy baseline (0.5933), confirming that *without delivery delay, the model is barely more informative than always predicting 5 stars*. Delivery delay carries most of the signal in the feature set.

3.5. Check 4: Data Reduction

The 50% data reduction check trains the model on a random half of the training set to assess whether performance depends critically on having the full training history. A model that performs identically with half the data is either learning a simple, data-efficient pattern or (more concerning) has already saturated its capacity with limited data.

The 50% model achieves 0.6209 accuracy (no balanced class weights, $n = 200$ trees), similar to the seed-sweep results. This indicates that the model is not constrained by training data volume for the current feature set: adding more historical orders does not meaningfully improve accuracy. The learning curve has plateaued, suggesting that the remaining predictive uncertainty comes from missing features (review text, product quality) rather than insufficient data.

Part II

Conclusions, Limitations, and Ethics

Chapter 4

Limitations and Future Work

Intellectual honesty requires documenting not only what a study achieves but what it does not and why. This chapter enumerates the specific limitations of the data, modelling choices, and statistical methodology applied in this project, and proposes concrete future improvements for each category.

4.1. Data Limitations

4.1.1. Absence of Customer Demographics

The Olist dataset contains no demographic information: no age, gender, income level, or education. As a result, the RFM segmentation is purely behavioural. It cannot answer questions such as “do younger customers give systematically lower review scores?” or “are high-income customers more tolerant of delays?” Demographic enrichment would substantially increase the depth of the segmentation analysis.

4.1.2. State-Level Geographic Resolution Only

The primary geographic feature in the current models is the customer’s state (`customer_state`), which is a very coarse proxy for the actual seller-to-customer shipping distance. Brazil’s twenty-six states vary enormously in geographic extent: São Paulo state alone is larger than the United Kingdom. Two customers in the same state might be 100 km apart (same metropolitan area) or 600 km apart (opposite sides of the state).

The `geolocation` table provides zip-code-level coordinates, enabling the computation of Haversine distances.¹ This feature is the single highest-priority improvement because geographic distance is a direct causal driver of delivery time.

¹The Haversine formula computes the great-circle distance between two points on a sphere: $d = 2r \arcsin\left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}}\right)$, where ϕ is latitude, λ is longitude, and r is the Earth’s radius. Implementation in Python: <https://pypi.org/project/haversine/>.

4.1.3. Single-Platform, Single-Country Dataset

All observations come from one platform (Olist) in one country (Brazil) during a 25-month window (2016–2018). Findings may not generalise to other platforms, countries, or time periods. The COVID-19 pandemic (2020 onwards) dramatically restructured e-commerce logistics globally, and the patterns observed in pre-pandemic Brazil may differ substantially from current behaviour.

4.1.4. Right-Censoring in the Final Months

Orders placed in the final weeks of the dataset window (September–October 2018) may not have received reviews by the time of the data extract. This creates right-censoring: the most recent orders are systematically under-represented in the review data, which slightly biases any trend analyses of review score over time.

4.2. Modelling Limitations

4.2.1. No Hyperparameter Tuning

The Random Forest and XGBoost models in this study use manually chosen hyperparameters rather than a formal search procedure (grid search, random search, or Bayesian optimisation).² Tuning on a validation fold (temporally contiguous with the training set, preceding the test set) would likely improve XGBoost’s macro F1 considerably, since the current `max_depth` and `learning_rate` values are educated guesses rather than data-driven choices.

4.2.2. Integer Encoding of Categorical Features

Integer codes for `product_category` and `customer_state` lose semantic structure. Target encoding³ or entity embeddings⁴ would give the model richer categorical representations.

4.2.3. Heuristic RFM Segment Labels

The twelve RFM segment names (*Champions*, *At Risk*, etc.) are industry conventions, not empirically derived from the data. The segment boundaries (e.g., the requirement that *Champions* must have $r \geq 4, f \geq 4, m \geq 4$) are arbitrary rules that may not correspond to the natural cluster structure of the Olist customer base. The K-Means comparison in provides a partial validation, but a fully data-driven segmentation using K-Means or Gaussian Mixture Models would be more defensible.

²Bayesian hyperparameter optimisation using `optuna` is an efficient alternative to grid search: <https://optuna.org>. A tutorial is at <https://towardsdatascience.com/hyperparameter-optimization-with-optuna-45813ae3a64d>.

³Target encoding replaces each category level with a regularised estimate of the mean target value for that level, computed within each training fold. It is supported in `scikit-learn` via `TargetEncoder`: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.TargetEncoder.html>.

⁴Entity embeddings learn a low-dimensional dense representation of each category level using a neural network. They were shown to outperform one-hot encoding for tabular categorical features in <https://arxiv.org/abs/1604.06737> (Guo and Berkahn, 2016).

4.2.4. No Review Text Features (NLP)

Review comment text is available for approximately 40% of orders. Incorporating text features would almost certainly improve review-score prediction significantly, because the text often directly states the reason for the rating (“the product arrived broken,” “delivery was very fast”). This is the highest-priority modelling improvement.

4.3. Statistical Limitations

4.3.1. Bootstrap IID Assumption

The bootstrap confidence intervals assume that test-set instances are independently and identically distributed. This assumption is partially violated: a customer who placed two orders in the test set contributes two correlated observations. A clustered bootstrap (sampling by `customer_unique_id` rather than by individual order) would produce wider, more honest confidence intervals.

4.4. Future Work

The following seven improvements are prioritised by expected impact:

1. **Haversine distance feature.** Compute exact zip-code-level seller-to-customer distance using the geolocation table. Expected to substantially improve both Task A and Task B performance.
2. **NLP on review text.** Train a DistilBERT text classifier on the 40% of reviews with comment text to dramatically improve review-score prediction quality.
3. **Seller-level features.** Aggregate dispatch delay, on-time delivery rate, and average review score per seller and attach these to each order as seller reputation features.
4. **BG/NBD Customer Lifetime Value model.** Replace the static RFM snapshot with a probabilistic lifetime model that predicts each customer’s future purchase probability and expected spend.
5. **Nested cross-validation with temporal folds.** Replace the single time split with a walk-forward cross-validation scheme to obtain lower-variance performance estimates and enable hyperparameter tuning.
6. **SMOTE oversampling for Task A.** Address the extreme class imbalance in late-delivery prediction (only $\approx 9\%$ late) using SMOTE or class-weighting strategies to improve recall on the minority class.
7. **FastAPI deployment.** Wrap the trained XGBoost model in a REST API endpoint using FastAPI,⁵ enabling real-time review-score risk prediction for newly placed orders in a production environment.

⁵FastAPI is a modern Python framework for building REST APIs with automatic OpenAPI documentation: <https://fastapi.tiangolo.com>.

Chapter 5

Ethics, Fairness, and Deployment Constraints

A machine learning model that is accurate on average can still be harmful if its errors are systematically concentrated on specific groups of people or businesses. The deployment of any model in a commercial context brings obligations beyond accuracy: the model must be fair, its predictions must not constitute discriminatory actions, and its limitations must be transparently communicated to decision-makers. This chapter addresses these obligations for the models developed in this project.

5.1. Privacy and Anonymisation

The Olist dataset has been designed with privacy as a primary constraint. All personally identifiable information has been removed at the source:

- Customer names, addresses, phone numbers, and email addresses are absent from every table.
- Customer and order identifiers (`customer_id`, `customer_unique_id`, `order_id`) are random hash strings generated by Olist, not derived from any personal attribute.
- Seller information is similarly anonymised: seller addresses are replaced with state-level aggregates.

Re-identification risk is effectively zero: there is no auxiliary information in the dataset that could be combined with the hashed identifiers to recover an individual's identity. The geolocation table contains zip-code-level coordinates, which in principle could be used to narrow a customer's location; however, zip codes in Brazil cover areas ranging from a few blocks (in dense urban centres) to entire municipalities (in rural areas), so location precision is insufficient for individual identification.

5.2. Differential Error Rates and Algorithmic Fairness

An accuracy figure of 62% tells us about the *average* performance of the review-score model. A fairness audit asks: is the error rate uniform across customer states, product categories, and seller states, or is it

systematically worse for specific groups?¹

The model's accuracy varies substantially by product category: from 37.5% for `office_furniture` to above 65% for high-volume, well-represented categories. Similarly, the state-level error analysis reveals that states with fewer orders in the training set tend to have higher error rates in the test set, because the model has seen fewer examples of purchase patterns from those states.

5.3. Deployment Constraints and the Model Card

Following the Model Card framework,² the following constraints must be respected in any deployment of the models developed in this project:

Intended use. The review-score prediction model is intended as a *risk-stratification tool*: it should flag orders with predicted low review scores for proactive seller outreach or logistics intervention. It is *not* intended to generate official customer satisfaction scores, to automatically penalise sellers, or to replace post-delivery review collection.

Known failure modes. The model fails disproportionately on small-volume product categories and on states with few training examples. Its accuracy drops from 62% to below 50% for several large categories (see Table 2.1).

Temporal validity. The model was trained on data from 2016–2017. Its performance should be monitored and the model retrained if the distribution of order values, delivery times, or review scores drifts materially. A monthly KS-test³ on feature distributions is recommended as a drift-detection mechanism.

Human oversight. No automated action (seller suspension, order cancellation) should be triggered solely by the model's output. Model predictions must be reviewed by a human operator before being used to make consequential decisions.

Feedback loops. If the model's predictions are used to trigger interventions (e.g., proactive customer service contact for flagged orders), and if those interventions improve outcomes, then future reviews for flagged orders will systematically differ from the training distribution. The model must be retrained on post-intervention data with the intervention indicator included as a feature.

¹Algorithmic fairness is an active research area. The seminal work by gebru2021 argues for documenting the demographic and geographic limitations of every model. A comprehensive overview of fairness metrics is at <https://fairmlbook.org>.

²Model Cards were proposed by Mitchell et al. (2019) as a standardised way to document the intended use, performance characteristics, and ethical constraints of ML models. Original paper: <https://arxiv.org/abs/1810.03993>. A practical template is at <https://modelcards.withgoogle.com>.

³The Kolmogorov–Smirnov (KS) test compares two distributions non-parametrically. A significant KS statistic in feature distributions between training and live data is an early warning of distribution shift. See https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html.

5.4. Responsible Use of RFM Segments

The RFM segmentation assigns labels such as “Lost” and “Hibernating” to specific customer cohorts. These labels are for internal business strategy use only. Several ethical constraints apply:

1. Customers must never be directly told which segment they belong to, particularly negatively labelled segments.
2. Marketing differentiation based on RFM segments (e.g., different discount levels) should comply with Brazilian consumer protection law (Código de Defesa do Consumidor) and must not constitute price discrimination against protected groups.
3. Segment membership is a snapshot in time. A customer classified as “Lost” today may re-engage tomorrow; segment labels must not create permanent internal stigma that prevents re-engagement outreach.

Chapter 6

Conclusions

6.1. Summary of Findings

This study applied a full end-to-end data science pipeline to the Olist Brazilian E-Commerce dataset, producing findings across three research questions:

Customer Segmentation (RQ1). RFM analysis of 93,358 unique customers identified twelve behavioural segments. The most important finding is the dominance of the *At Risk* segment: 15,490 customers (16.6% of the base) who account for 23.66% of total platform revenue but whose last purchase was, on average, 381 days ago. The *Champions* segment (6,474 customers, 6.9% of the base) contributes 13.42% of revenue at an average lifetime spend of R\$ 274.16. The low average frequency (close to 1.0 across all segments) reveals that Olist operated primarily as a single-purchase acquisition engine during the study period rather than a loyalty-building platform.

Delivery Performance (RQ2). All top product categories are delivered well ahead of schedule (average delay of -11 to -12 days), reflecting Olist's conservative estimation strategy. The binary late-delivery Random Forest achieves 0.9226 accuracy, but a macro F1 of only 0.4799 reveals the challenge of detecting the minority late-delivery class. The most predictive features for delivery outcomes are product weight, geographic origin–destination pair, and the number of items in the order.

Review Score Prediction (RQ3). XGBoost is the best model for five-class review score prediction, achieving 0.6238 accuracy, a statistically significant improvement over the 0.5933 majority-class baseline (non-overlapping 95% CIs). Delivery delay is the dominant predictor: SHAP values confirm that high delay strongly pushes predictions towards low review scores, consistent with the bucket analysis showing a collapse from 4.32 stars (very early delivery) to 1.72 stars (very late delivery). Despite this, 38% of review score variance remains unexplained by order-level and delivery features alone.

6.2. Hypothesis Verdicts

H1 (Pareto Revenue) — *Partially supported.* Champions and Loyal customers are not a small, dominant revenue elite: together they represent 15.67% of customers and 21.28% of revenue, a modest concentration. The more striking finding is that the At Risk segment accounts for the largest single revenue share (23.66%). The classical 80/20 Pareto rule does not hold; the revenue distribution is more uniform than expected.

H2 (Delay Causes Dissatisfaction) — *Strongly supported.* The Spearman correlation between delivery delay and review score is significantly negative. The bucket analysis reveals a dramatic, non-linear collapse in satisfaction at the zero-delay threshold: orders arriving one or more days late average 2.72 stars compared with 4.32 stars for early deliveries. H2 is the most robustly confirmed finding in this study.

H3 (Models Beat Baseline) — *Supported for XGBoost; not supported for RF accuracy.* XGBoost achieves 0.6238 accuracy with a CI that does not overlap with the dummy's (0.5933, CI [0.5889, 0.5975]). Random Forest achieves lower accuracy than the dummy due to balanced class weights, but achieves higher macro F1 (0.2924 vs. ≈ 0.17), suggesting it has learned to identify minority classes. The effect size is small (Cohen's $h \approx 0.06$).

H4 (Delivery Delay Dominates) — *Strongly supported.* Both Gini impurity importance and SHAP values identify `delivery_delay_days` as the top feature by a large margin. Permutation importance and the feature ablation check (which drops accuracy to near-baseline when delivery delay is removed) provide convergent, multi-method confirmation.

6.3. Three Actionable Business Recommendations

1. **Launch a targeted At-Risk re-engagement programme.** The 15,490 customers in the At Risk segment hold R\$ 3,127,833 in historical revenue. A personalised win-back campaign with a category-specific discount of 10–15% and an automated email sequence, reaching even 10% of this cohort, would recover approximately R\$ 313,000 in incremental annual revenue. This is the highest-ROI single action identified by the RFM analysis.
2. **Prioritise logistics intervention for orders with predicted delay.** Deploy the late-delivery Random Forest in real-time at order placement to flag high-risk orders (heavy products, distant seller–buyer pairs, high freight ratio). Use these flags to trigger proactive seller outreach, carrier pre-positioning, or customer expectation-adjustment messaging. Even a 20% reduction in the late-delivery rate would shift a meaningful fraction of 2-star and 3-star reviews towards 4 and 5 stars.
3. **Set conservative delivery estimates as a systematic policy.** The data shows that orders arriving 1–7 days late average 2.72 stars, while orders arriving more than 7 days early average 4.32 stars.

Olist already quotes conservative estimates (most orders arrive ~11 days before the estimate), but formalising this as an explicit policy, and extending it to the sellers with the highest late-delivery rates, would reduce the frequency of below-threshold outcomes at low cost.

References

- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., . . . Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the conference on fairness, accountability, and transparency* (pp. 220–229). ACM. doi: 10.1145/3287560.3287596
- Raschka, S. (2018). *Model evaluation, model selection, and algorithm selection in machine learning*. arXiv preprint arXiv:1811.12808. Retrieved from <https://arxiv.org/abs/1811.12808>